In [2]:

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    roc_auc_score
)
```

In [3]:

```
df = pd.read_csv("WA_Fn-UseC_-HR-Employee-Attrition.csv")

print(df.head())
print(df.shape)
```

```
   Age Attrition     BusinessTravel  DailyRate              Department  \
0   41       Yes      Travel_Rarely       1102                   Sales
1   49        No  Travel_Frequently        279  Research & Development
2   37       Yes      Travel_Rarely       1373  Research & Development
3   33        No  Travel_Frequently       1392  Research & Development
4   27        No      Travel_Rarely        591  Research & Development

   DistanceFromHome  Education EducationField  EmployeeCount  EmployeeNumber  \
0                 1          2  Life Sciences              1               1
1                 8          1  Life Sciences              1               2
2                 2          2          Other              1               4
3                 3          4  Life Sciences              1               5
4                 2          1        Medical              1               7

   ...  RelationshipSatisfaction StandardHours  StockOptionLevel  \
0  ...                         1           80                 0
1  ...                         4           80                 1
2  ...                         2           80                 0
3  ...                         3           80                 0
4  ...                         4           80                 1

   TotalWorkingYears  TrainingTimesLastYear WorkLifeBalance  YearsAtCompany  \
0                  8                      0               1               6
1                 10                      3               3              10
2                  7                      3               3               0
3                  8                      3               3               8
4                  6                      3               3               2

  YearsInCurrentRole  YearsSinceLastPromotion  YearsWithCurrManager
0                  4                        0                     5
1                  7                        1                     7
2                  0                        0                     0
```

```
3                  7                        3                       0
4                  2                        2                       2

[5 rows x 35 columns]
(1470, 35)
```

In [4]:

```
print(df.info())
print(df.describe())
print(df.isnull().sum())
print(df['Attrition'].value_counts())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1470 entries, 0 to 1469
Data columns (total 35 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Age                       1470 non-null   int64
 1   Attrition                 1470 non-null   object
 2   BusinessTravel            1470 non-null   object
 3   DailyRate                 1470 non-null   int64
 4   Department                1470 non-null   object
 5   DistanceFromHome          1470 non-null   int64
 6   Education                 1470 non-null   int64
 7   EducationField            1470 non-null   object
 8   EmployeeCount             1470 non-null   int64
 9   EmployeeNumber            1470 non-null   int64
 10  EnvironmentSatisfaction   1470 non-null   int64
 11  Gender                    1470 non-null   object
 12  HourlyRate                1470 non-null   int64
 13  JobInvolvement            1470 non-null   int64
 14  JobLevel                  1470 non-null   int64
 15  JobRole                   1470 non-null   object
 16  JobSatisfaction           1470 non-null   int64
 17  MaritalStatus             1470 non-null   object
 18  MonthlyIncome             1470 non-null   int64
 19  MonthlyRate               1470 non-null   int64
 20  NumCompaniesWorked        1470 non-null   int64
 21  Over18                    1470 non-null   object
 22  OverTime                  1470 non-null   object
 23  PercentSalaryHike         1470 non-null   int64
 24  PerformanceRating         1470 non-null   int64
 25  RelationshipSatisfaction  1470 non-null   int64
 26  StandardHours             1470 non-null   int64
 27  StockOptionLevel          1470 non-null   int64
 28  TotalWorkingYears         1470 non-null   int64
 29  TrainingTimesLastYear     1470 non-null   int64
 30  WorkLifeBalance           1470 non-null   int64
 31  YearsAtCompany            1470 non-null   int64
 32  YearsInCurrentRole        1470 non-null   int64
 33  YearsSinceLastPromotion   1470 non-null   int64
 34  YearsWithCurrManager      1470 non-null   int64
dtypes: int64(26), object(9)
memory usage: 402.1+ KB
None
                Age    DailyRate  DistanceFromHome    Education  EmployeeCount  \
count  1470.000000  1470.000000       1470.000000  1470.000000         1470.0
mean     36.923810   802.485714          9.192517     2.912925            1.0
std       9.135373   403.509100          8.106864     1.024165            0.0
```

```
min      18.000000   102.000000          1.000000     1.000000            1.0
25%      30.000000   465.000000          2.000000     2.000000            1.0
50%      36.000000   802.000000          7.000000     3.000000            1.0
75%      43.000000  1157.000000         14.000000     4.000000            1.0
max      60.000000  1499.000000         29.000000     5.000000            1.0

       EmployeeNumber  EnvironmentSatisfaction   HourlyRate  JobInvolvement  \
count     1470.000000              1470.000000  1470.000000     1470.000000
mean      1024.865306                 2.721769    65.891156        2.729932
std        602.024335                 1.093082    20.329428        0.711561
min          1.000000                 1.000000    30.000000        1.000000
25%        491.250000                 2.000000    48.000000        2.000000
50%       1020.500000                 3.000000    66.000000        3.000000
75%       1555.750000                 4.000000    83.750000        3.000000
max       2068.000000                 4.000000   100.000000        4.000000

          JobLevel  ...  RelationshipSatisfaction  StandardHours  \
count  1470.000000  ...               1470.000000         1470.0
mean      2.063946  ...                  2.712245           80.0
std       1.106940  ...                  1.081209            0.0
min       1.000000  ...                  1.000000           80.0
25%       1.000000  ...                  2.000000           80.0
50%       2.000000  ...                  3.000000           80.0
75%       3.000000  ...                  4.000000           80.0
max       5.000000  ...                  4.000000           80.0

       StockOptionLevel  TotalWorkingYears  TrainingTimesLastYear  \
count       1470.000000        1470.000000            1470.000000
mean           0.793878          11.279592               2.799320
std            0.852077           7.780782               1.289271
min            0.000000           0.000000               0.000000
25%            0.000000           6.000000               2.000000
50%            1.000000          10.000000               3.000000
75%            1.000000          15.000000               3.000000
max            3.000000          40.000000               6.000000

       WorkLifeBalance  YearsAtCompany  YearsInCurrentRole  \
count      1470.000000     1470.000000         1470.000000
mean          2.761224        7.008163            4.229252
std           0.706476        6.126525            3.623137
min           1.000000        0.000000            0.000000
25%           2.000000        3.000000            2.000000
50%           3.000000        5.000000            3.000000
75%           3.000000        9.000000            7.000000
max           4.000000       40.000000           18.000000

       YearsSinceLastPromotion  YearsWithCurrManager
count              1470.000000           1470.000000
mean                  2.187755              4.123129
std                   3.222430              3.568136
min                   0.000000              0.000000
25%                   0.000000              2.000000
50%                   1.000000              3.000000
75%                   3.000000              7.000000
max                  15.000000             17.000000

[8 rows x 26 columns]
Age                       0
Attrition                 0
```

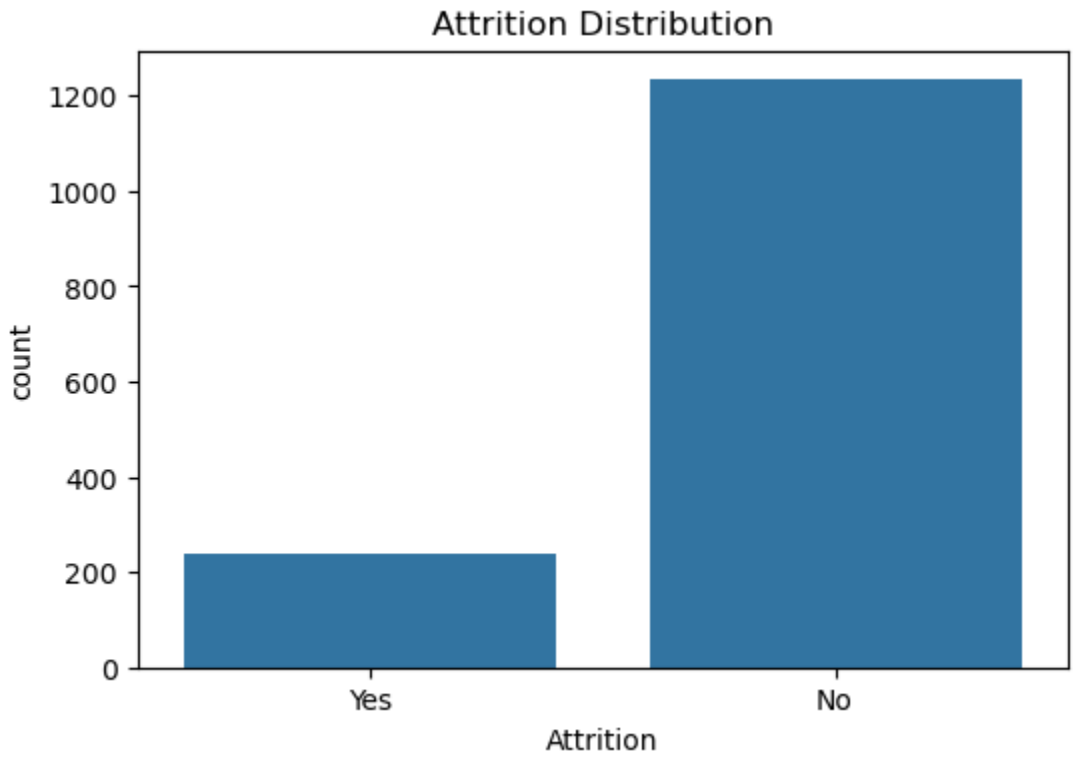
```
BusinessTravel              0
DailyRate                   0
Department                  0
DistanceFromHome            0
Education                   0
EducationField              0
EmployeeCount               0
EmployeeNumber              0
EnvironmentSatisfaction     0
Gender                      0
HourlyRate                  0
JobInvolvement              0
JobLevel                    0
JobRole                     0
JobSatisfaction             0
MaritalStatus               0
MonthlyIncome               0
MonthlyRate                 0
NumCompaniesWorked          0
Over18                      0
OverTime                    0
PercentSalaryHike           0
PerformanceRating           0
RelationshipSatisfaction    0
StandardHours               0
StockOptionLevel            0
TotalWorkingYears           0
TrainingTimesLastYear       0
WorkLifeBalance             0
YearsAtCompany              0
YearsInCurrentRole          0
YearsSinceLastPromotion     0
YearsWithCurrManager        0
dtype: int64
Attrition
No     1233
Yes     237
Name: count, dtype: int64
```

In [5]:

```
drop_cols = [
    'EmployeeCount',
    'EmployeeNumber',
    'Over18',
    'StandardHours'
]
df.drop(columns=drop_cols, inplace=True)
```

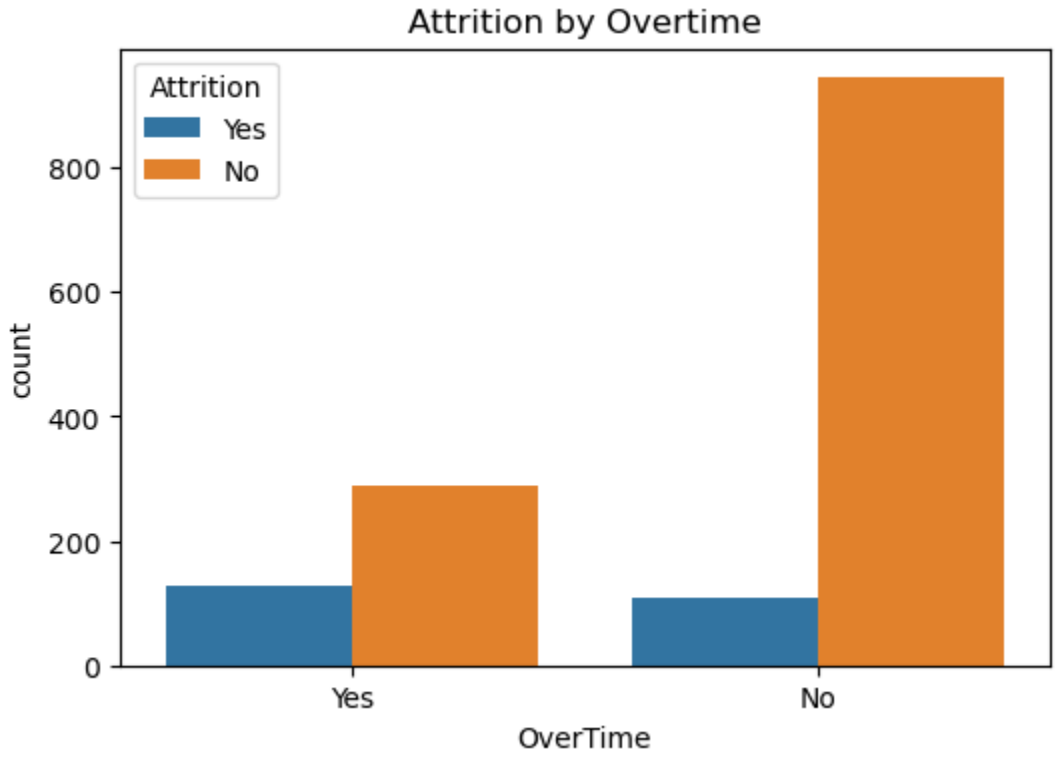
In [6]:

```
plt.figure(figsize=(6,4))
sns.countplot(x='Attrition', data=df)
plt.title("Attrition Distribution")
plt.show()
```



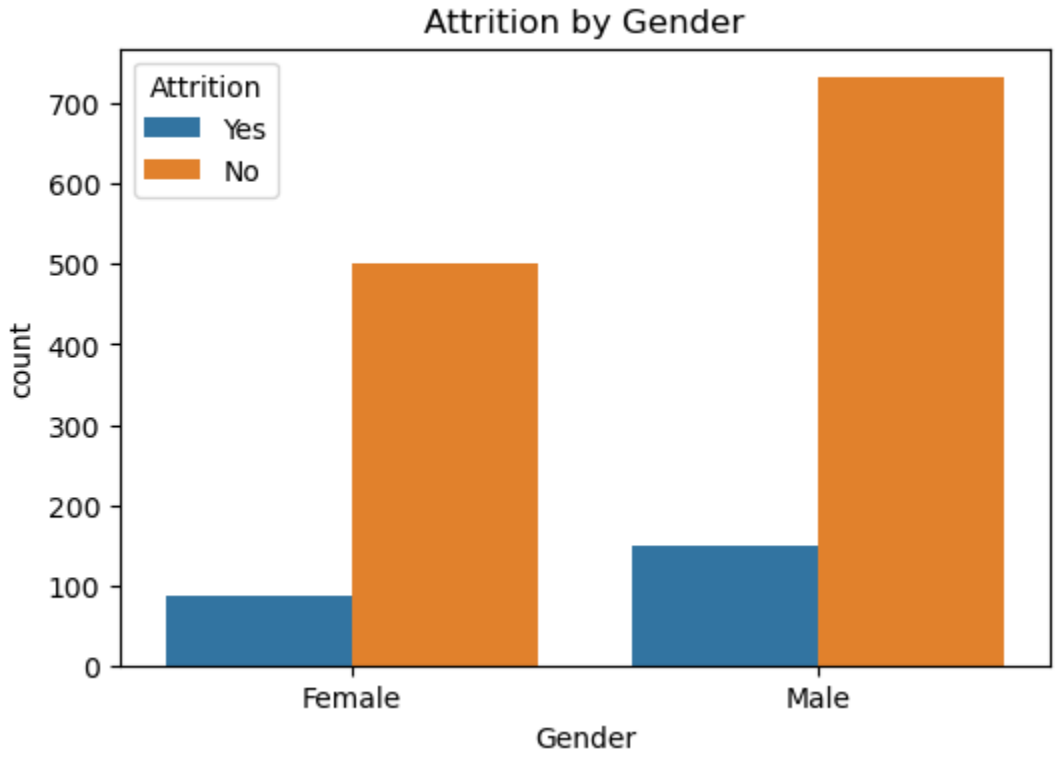
In [7]:

```
plt.figure(figsize=(6,4))
sns.countplot(
    x='OverTime',
    hue='Attrition',
    data=df
)
plt.title("Attrition by Overtime")
plt.show()
```



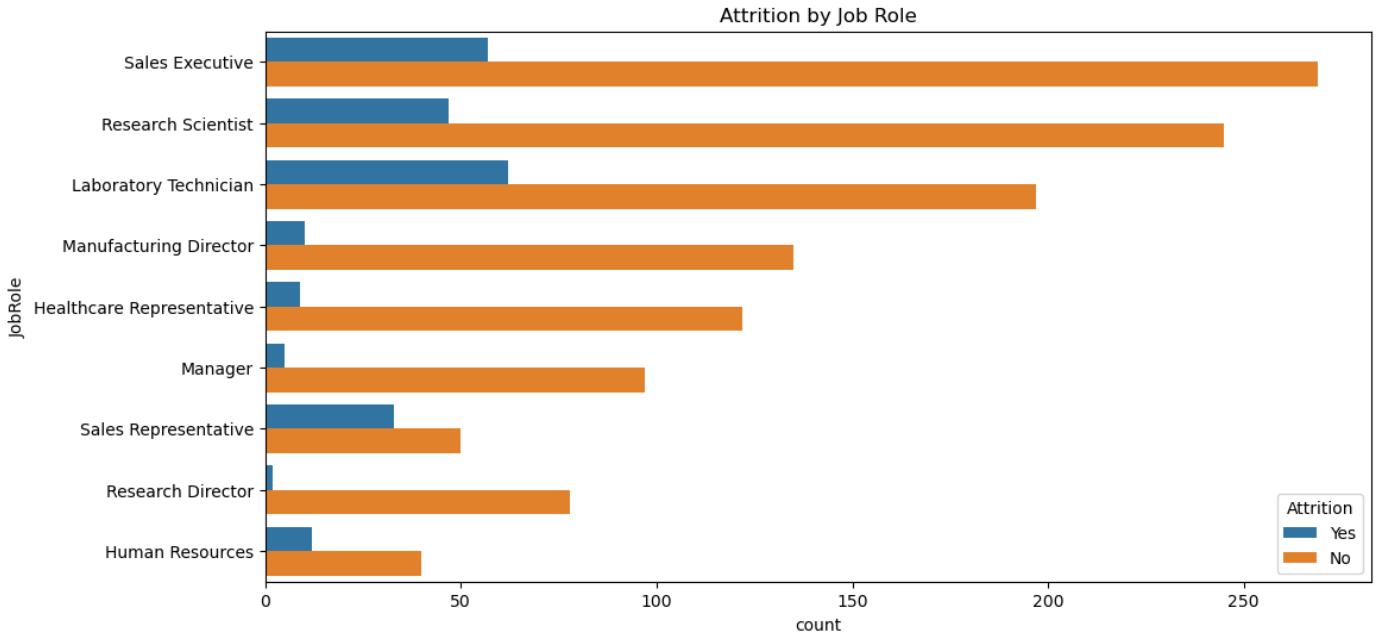
In [8]:

```
plt.figure(figsize=(6,4))
sns.countplot(
    x='Gender',
    hue='Attrition',
    data=df
)
plt.title("Attrition by Gender")
plt.show()
```



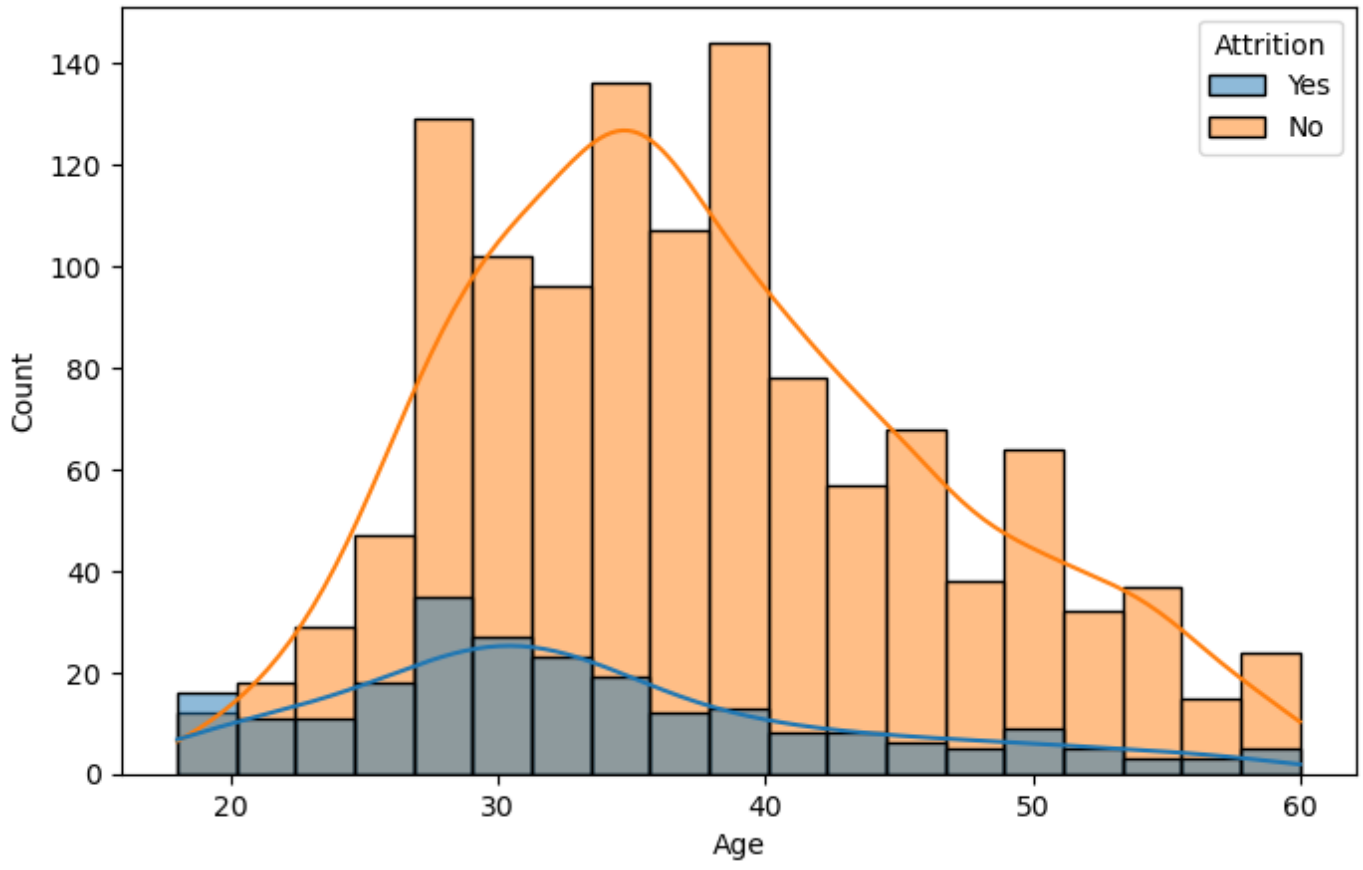
In [9]:

```
plt.figure(figsize=(12,6))
sns.countplot(
    y='JobRole',
    hue='Attrition',
    data=df
)
plt.title("Attrition by Job Role")
plt.show()
```



In [10]:

```
plt.figure(figsize=(8,5))
sns.histplot(
    data=df,
    x='Age',
    hue='Attrition',
    kde=True
)
plt.show()
```



In [11]:

```
df['Attrition'] = df['Attrition'].map({
    'Yes':1,
    'No':0
})
```

In [12]:

```
df_encoded = pd.get_dummies(
    df,
    drop_first=True
)
```

In [13]:

```
X = df_encoded.drop('Attrition', axis=1)
y = df_encoded['Attrition']
```

In [14]:

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
```

In [15]:

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

In [16]:

```
lr = LogisticRegression(max_iter=1000)
lr.fit(X_train_scaled, y_train)
y_pred_lr = lr.predict(X_test_scaled)
```

In [17]:

```
lr_accuracy = accuracy_score(
    y_test,
    y_pred_lr
)
print("Logistic Regression Accuracy:", lr_accuracy)
```

```
Logistic Regression Accuracy: 0.8605442176870748
```

In [18]:

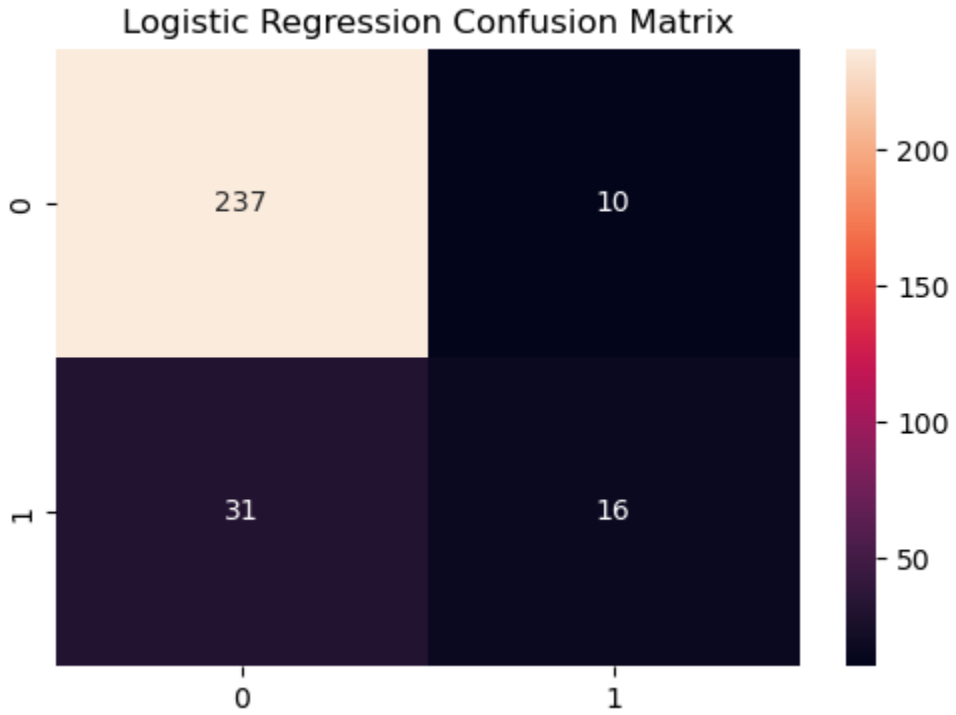
```
print(classification_report(
    y_test,
    y_pred_lr
))
```

```
              precision    recall  f1-score   support

           0       0.88      0.96      0.92       247
           1       0.62      0.34      0.44        47

    accuracy                           0.86       294
   macro avg       0.75      0.65      0.68       294
```

```
weighted avg       0.84      0.86      0.84       294
```

In [22]:

```
cm = confusion_matrix(
    y_test,
    y_pred_lr
)
plt.figure(figsize=(6,4))
sns.heatmap(
    cm,
    annot=True,
    fmt='d'
)
plt.title("Logistic Regression Confusion Matrix")
plt.show()
```



In [23]:

```
y_prob = lr.predict_proba(X_test_scaled)[:,1]
roc = roc_auc_score(
    y_test,
    y_prob
)
print("ROC-AUC Score:", roc)
```

```
ROC-AUC Score: 0.8079076578516668
```

In [24]:

```
dt = DecisionTreeClassifier(
    max_depth=5,
    random_state=42
)
dt.fit(X_train, y_train)
y_pred_dt = dt.predict(X_test)
```

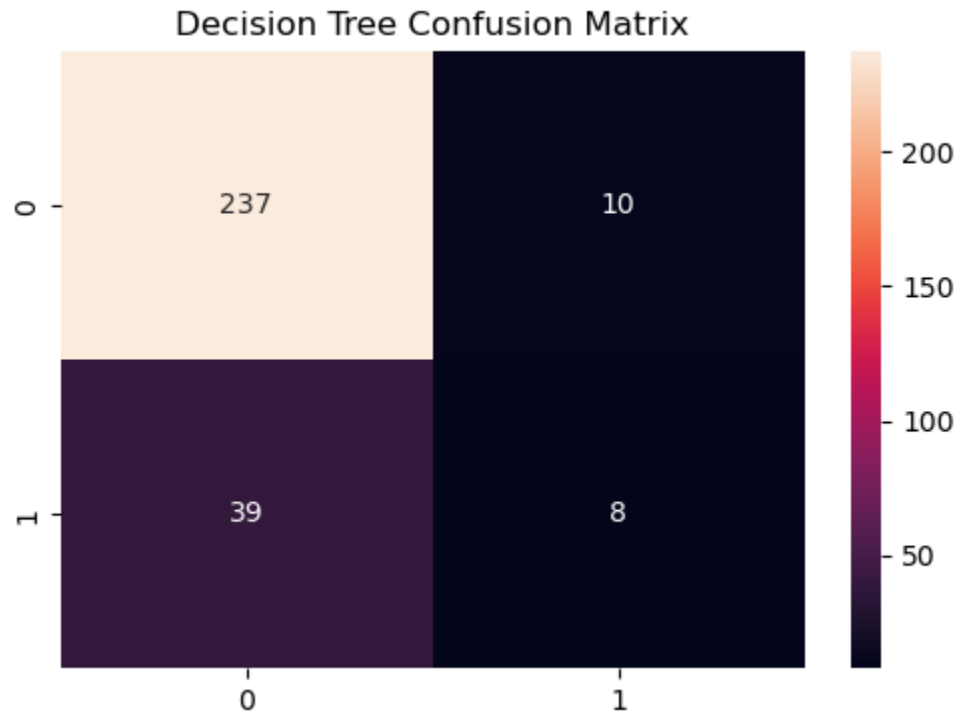
In [25]:

```
dt_accuracy = accuracy_score(
    y_test,
    y_pred_dt
)
print("Decision Tree Accuracy:", dt_accuracy)
print(classification_report(
    y_test,
    y_pred_dt
))
```

```
Decision Tree Accuracy: 0.8333333333333334
              precision    recall  f1-score   support

           0       0.86      0.96      0.91       247
           1       0.44      0.17      0.25        47

    accuracy                           0.83       294
   macro avg       0.65      0.56      0.58       294
weighted avg       0.79      0.83      0.80       294
```

In [26]:

```
cm_dt = confusion_matrix(
    y_test,
    y_pred_dt
)
plt.figure(figsize=(6,4))
sns.heatmap(
    cm_dt,
    annot=True,
    fmt='d'
)
plt.title("Decision Tree Confusion Matrix")
plt.show()
```



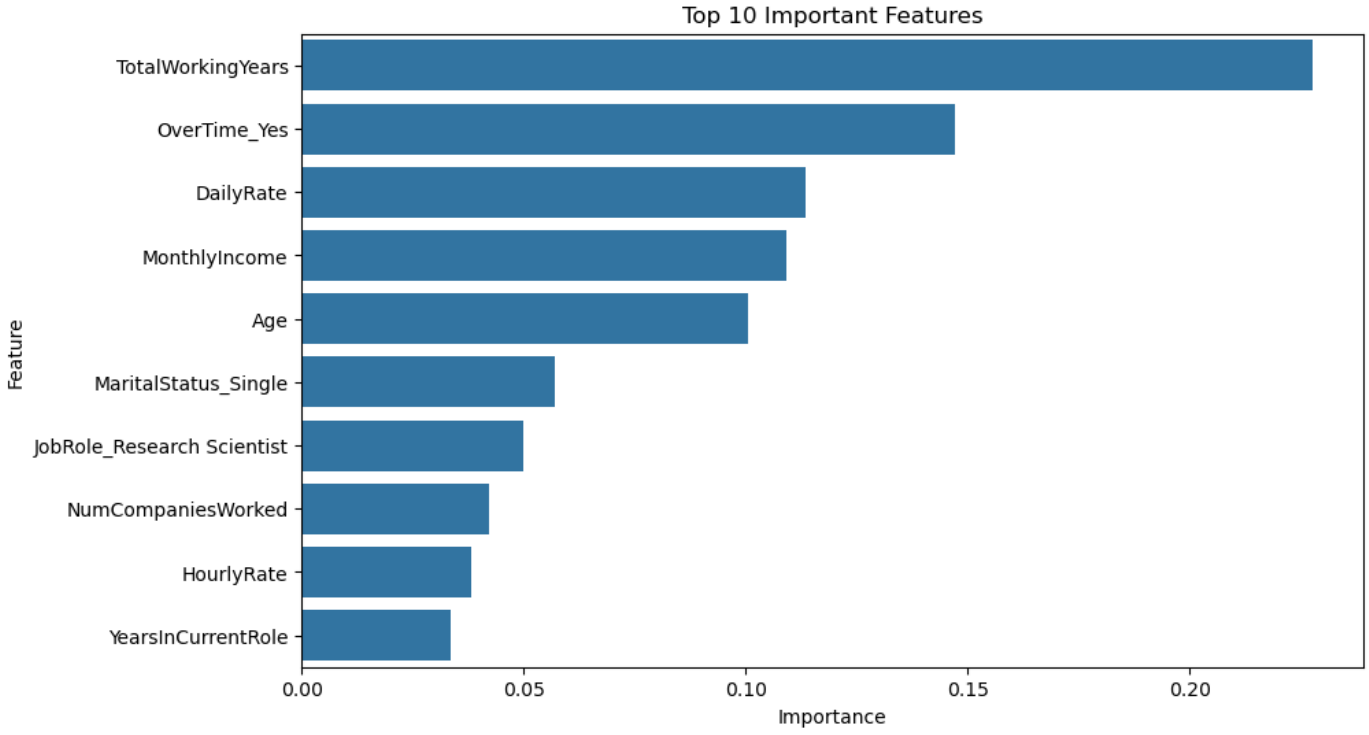
In [27]:

```
importance = pd.DataFrame({
    'Feature': X.columns,
    'Importance': dt.feature_importances_
})
importance = importance.sort_values(
    by='Importance',
    ascending=False
)
print(importance.head(10))
```

```
                       Feature  Importance
16           TotalWorkingYears    0.227763
43                OverTime_Yes    0.147191
1                    DailyRate    0.113395
9                MonthlyIncome    0.108999
0                          Age    0.100578
42        MaritalStatus_Single    0.057004
38  JobRole_Research Scientist    0.049864
11          NumCompaniesWorked    0.042111
5                   HourlyRate    0.038135
20          YearsInCurrentRole    0.033556
```

In [28]:

```
plt.figure(figsize=(10,6))
sns.barplot(
    x='Importance',
    y='Feature',
    data=importance.head(10)
)
plt.title("Top 10 Important Features")
plt.show()
```



In [29]:

```
all_prob = lr.predict_proba(
    scaler.transform(X)
```

```
)[:,1]
df['Risk_Score'] = all_prob
```

In [32]:

```
def risk_category(score):
    if score >= 0.70:
        return "High Risk"
    elif score >= 0.40:
        return "Medium Risk"
    return "Low Risk"

df['Risk_Category'] = df['Risk_Score'].apply(risk_category)
```

In [33]:

```
print(df[['Risk_Score', 'Risk_Category']].head())
print(df['Risk_Category'].value_counts())
```

```
   Risk_Score Risk_Category
0    0.726492     High Risk
1    0.015506      Low Risk
2    0.432791   Medium Risk
3    0.100361      Low Risk
4    0.416234   Medium Risk
Risk_Category
Low Risk       1278
Medium Risk     128
High Risk        64
Name: count, dtype: int64
```

In [34]:

```
df.to_csv(
    "Employee_Attrition_Predictions.csv",
    index=False
)

print("File Exported Successfully")
```

```
File Exported Successfully
```